AREST: Applicative REpresentational State Transfer

Composing Backus's AST with Fielding's REST
for Specification-Equivalent Application Frameworks

Samuel Lippert
Drivly Inc

Abstract

We present AREST (**A**pplicative **RE**presentational **S**tate **T**ransfer), a compositional architecture that unifies Backus's Applicative State Transition systems (1978) with Fielding's Representational State Transfer (2000). Domain knowledge is expressed as first-order logic propositions in controlled natural language (FORML 2 readings), compiled into executable functions via Backus's combining forms, and rendered as hypermedia representations with state machine transitions as navigation links. The representational state being transferred is the applicative state being transformed. We demonstrate that this composition eliminates the specification–implementation gap: the natural language reading is simultaneously the specification, the executable, and the proof obligation.

1 Introduction

Application development produces two artifacts, a specification and an implementation, that diverge over time. The specification states "each customer has exactly one email"; the implementation enforces this through database constraints, API validation, frontend checks, and tests. These are four independent projections of the same proposition that can disagree.

We observe that this divergence arises from treating specification and implementation as separate artifacts. If the sentence

Each Customer has exactly one Email.

is simultaneously a human-readable requirement and a machine-executable uniqueness–mandatory constraint, no translation step exists to introduce error.

AREST achieves this by composing two state transfer systems that operate at orthogonal layers:

Definition 1 (AST)**.** An *Applicative State Transition* system [1] consists of a set of objects O, a set of functions $F : O \to O$, and a single operation, application: $f : x$. The state $s \in O$ is transformed by applying a command: $s_{n+1} = f : s_n$. Programs are point-free compositions of combining forms. No variables are named; no state is mutated.

Definition 2 (REST)**.** A *Representational State Transfer* system (Fielding, 2000) communicates state between distributed components by transferring representations via a uniform interface. Hypermedia links in the representation encode valid state transitions (HATEOAS).

Definition 3 (AREST)**.** An *Applicative REpresentational State Transfer* system is the composition AST$\circ$REST: the representational state being transferred is the applicative state being transformed, rendered as hypermedia with transition links.

2 Theoretical Foundations

AREST synthesizes several theoretical traditions.

2.1 First-Order Predicate Logic

Domain knowledge in AREST is expressed as propositions in first-order logic (FOL):

$$\forall p \, \forall c_1 \, \forall c_2 \, (\text{born}(p, c_1) \wedge \text{born}(p, c_2) \rightarrow c_1 = c_2) \tag{1}$$

Equation (1) is verbalized as: "Each Person was born in at most one Country." The verbalization and the formula are isomorphic, the same proposition in two notations.

2.2 Lambda Calculus, FP Algebra, and AST

Church's lambda calculus [2] provides the computation model: a fully applied term (all roles bound) is a fact, a partially applied term is a query, and evaluation is beta reduction. Backus [1] lifted this to function-level programming: programs built from combining forms without naming variables.

Definition 4 (Combining Forms)**.** Let $f, g, f_1, \ldots, f_n$ be functions and x an object.

$$[f_1, \ldots, f_n] : x = \langle f_1 : x, \ldots, f_n : x \rangle \qquad \text{(Construction)} \tag{2}$$

$$(f \circ g) : x = f : (g : x) \qquad \text{(Composition)} \tag{3}$$

$$(\alpha f) : \langle x_1, \ldots, x_n \rangle = \langle f : x_1, \ldots, f : x_n \rangle \qquad \text{(Apply-to-All)} \tag{4}$$

$$(/f) : \langle x_1, \ldots, x_n \rangle = f : \langle x_1, /f : \langle x_2, \ldots \rangle \rangle \qquad \text{(Insert)} \tag{5}$$

$$(p \rightarrow f; g) : x = \begin{cases} f : x & \text{if } p : x = T \\ g : x & \text{if } p : x = F \end{cases} \qquad \text{(Condition)} \tag{6}$$

$$(\text{bu } f \; a) : x = f : \langle a, x \rangle \qquad \text{(Binary-to-Unary)} \tag{7}$$

In AREST, these forms have precise domain interpretations:

Backus Form	**AREST Interpretation**
Construction $[f_1, \ldots, f_n]$	Graph Schema (CONS of Roles)
Selector s	Role at position s
Composition $f \circ g$	Derivation rule chain
Apply-to-All αf	Population traversal
Insert $/f$	Aggregation (exists, forall)
Condition $p \rightarrow f; g$	Constraint evaluation, guards
Binary-to-Unary bu f x	Partial application (query)
Constant $\bar{x}$	Literal value in a reading

Backus's AST extends FP with explicit state: the system state is an object s, and a program p transforms it: $s' = p : s$. The system evolves by successive application. In AREST, the state is the population of facts, and each API request is a command applied to the population.

2.3 Object-Role Modeling

ORM2 [5] models information as elementary facts, where objects play roles in predicates. The surface syntax, FORML 2 [7], is a controlled natural language:

```
Organization(.Slug) is an entity type.
Organization has Name.
  Each Organization has exactly one Name.
```

Each sentence is a FOL proposition. The reference scheme `(.Slug)` defines the entity's identity predicate. The constraint `Each Organization has exactly one Name` combines a uniqueness constraint (UC, "at most one") with a mandatory constraint (MC, "at least one"). The CSDP (Conceptual Schema Design Procedure) [5] provides a 7-step procedure guaranteeing semantic completeness, and the RMAP (Relational Mapping Procedure) [5] maps conceptual schemas to relational schemas.

The key property: the verbalization is lossless. Every ORM2 constraint has exactly one canonical verbalization pattern, and every well-formed verbalization maps to exactly one constraint. The grammar is restrictive enough to parse deterministically.

ORM2 models state machines as facts about facts. A State Machine instance is a separate entity from the Resource it governs, connected by the fact type `State Machine is for Resource` (1:1). Objectification of such fact types requires a spanning uniqueness constraint [6].

RMAP maps conceptual schemas to Codd's relations [3]: uniqueness constraints become keys, mandatory constraints become NOT NULL. In AREST, each entity instance is an independent storage unit keyed by its reference scheme, satisfying third normal form.

2.4 REST

Fielding's architectural style [4] constrains distributed systems to transfer representations of state via a uniform interface. The critical constraint:

> *Hypermedia as the engine of application state* (HATEOAS): each response includes links encoding valid state transitions.

In AREST, the state machine's valid transitions from the current status serve as the HATEOAS links. The API response for an entity in status "Draft" includes actions `[place, cancel]`, which are transitions compiled from the readings.

3 The AREST Execution Model

3.1 Commands as Function Application

Every API interaction is a Command: a function from population to population.

$$\text{Command} : \text{Population} \to (\text{Population}', \text{Representation}) \tag{8}$$

A CreateEntity command is a composed function that extends the population with new formulae:

$$\text{create} = \text{emit} \circ \text{derive} \circ \text{validate} \circ \text{resolve} \tag{9}$$

where:

- resolve: applies the reference scheme selector to determine the entity's identity
- validate: evaluates constraints against the current population
- derive: forward-chains derivation rules to fixed point, adding derived formulae (including state machine initialization) to the population
- emit: produces the new population containing the entity, its state machine instance, and all derived facts

State machine initialization is not a separate step. The state machine definition is a set of formulae in the population. When the entity is created, the derivation rules produce the State Machine instance and its initial Status as derived facts. The composition is compiled from the readings. The engine applies it as one reduction.

3.2 State Machines as Fold

A state machine is Backus's Insert (fold) over an event stream:

$$\text{run_machine} = /\text{transition} : \langle e_1, \ldots, e_n \rangle \tag{10}$$

Each transition is a Condition:

$$\text{transition} = (p \to \bar{s'}; \text{id}) \tag{11}$$

where $p = \text{eq} \circ [\text{id}, \overline{\langle s_{\text{from}}, e \rangle}]$ checks whether the current state and event match, and $\bar{s'}$ is the constant function returning the target state.

The transition table compiles to nested conditions. The first matching condition fires; the rest fall through to identity (state unchanged):

$$f = (p_1 \to \bar{s_1};\ (p_2 \to \bar{s_2};\ \ldots;\ \text{id})) \tag{12}$$

Guards are constraints that must produce zero violations for the transition to fire:

$$\text{guarded_transition} = (\text{guard_passes} \wedge p \to \bar{s'};\ \text{id}) \tag{13}$$

where $\text{guard_passes} = \text{null} \circ \text{evaluate_constraints}$.

3.3 Derivation Rules as Policy

Any policy that can be expressed as a derivation rule is evaluated by the engine. Access control, visibility scoping, and domain evolution are all derivation rules compiled to Composition chains. For example, the access rule

```
User can access Domain iff User has Org Role
  in Organization and Domain belongs to
  that Organization.
```

compiles to:

$$\text{access} = /\text{or} \circ \alpha(\text{bu eq } d_{\text{org}}) : \text{user_orgs} \tag{14}$$

Forward chaining from the user's identity resolves the accessible domains. The derivation rule is the policy.

Domain evolution follows the same pattern. A proposed change to the readings is itself a fact in the population. A Curry-Howard analogy [8] applies: proposing a new fact type is analogous to proposing a theorem, and the schema is accepted only if the engine can verify its consistency. This is a structural parallel, not a formal type-theoretic isomorphism. CSDP validation serves as the consistency checker: just as a type checker rejects ill-typed programs, the engine rejects schemas that violate the design procedure's rules.

4 End-to-End Example

We trace a single domain through the full AREST pipeline. An order management domain is declared in FORML 2:

```
Order(.OrderId) is an entity type.
Customer(.Name) is an entity type.
Order was placed by Customer.
  Each Order was placed by exactly one Customer.
Order transitions:
  Draft -[Place]-> Placed -[Ship]-> Shipped -[Deliver]-> Delivered
```

The constraint compiles to FOL: $\forall o \forall c_1 \forall c_2\ (placedBy(o, c_1) \wedge placedBy(o, c_2) \rightarrow c_1 = c_2)$. The fact type compiles to a Construction of Selectors: $schema_{placedBy} = [sel_1, sel_2]$. The state machine compiles to nested Conditions (Section 3.2).

A client sends `POST /orders {"customer":"acme"}`. The engine applies $emit \circ derive \circ validate \circ resolve$: *resolve* determines identity from the reference scheme; *validate* checks the UC on *placedBy*; *derive* forward-chains to fixed point, creating a State Machine with initial Status = "Draft"; *emit* returns $\text{Pop}' = \text{Pop} \cup \{Order(\texttt{ord-1}),\ placedBy(\texttt{ord-1}, \texttt{acme}),\ SM(\texttt{ord-1}, \texttt{Draft})\}$. The response:

```
{ "id": "ord-1", "customer": "acme", "status": "Draft",
  "_links": { "place": { "href": "/orders/ord-1/place" } } }
```

The `_links` are projections of the compiled transition table. When the client follows "place," the engine applies $transition : \langle \texttt{Draft}, \texttt{Place} \rangle = \texttt{Placed}$ and the response includes new link `{ship}`, reflecting the updated valid transitions.

5 Properties

Theorem 1 (Specification Equivalence)**.** In AREST, the FORML 2 reading is isomorphic to the executable constraint. No translation step exists between specification and implementation.

Proof. Each FORML 2 sentence has exactly one canonical verbalization pattern mapping to exactly one FOL formula [7]. The parser produces a schema of propositions. The engine compiles them to combining forms. Evaluation is beta reduction. The transformation is deterministic and invertible: given the compiled form, the original reading can be recovered by verbalization. □

Theorem 2 (Completeness of State Transfer)**.** A CreateEntity command produces the complete new population in one function application: the resource entity, the state machine instance, derived facts, and constraint violations.

Proof. The command is a composition of functions. The composition is applied atomically; no intermediate state is observable. The result contains the complete new population. □

Corollary 1 (HATEOAS from Readings)**.** The hypermedia links in the REST representation are projections of the compiled state machine's transition table. No URL patterns are hardcoded.

6 Discussion

6.1 Bounding the Specification–Implementation Gap

Theorem 1 holds to the extent that the engine correctly compiles and evaluates. The gap does not vanish; it relocates from "specification vs. hand-written code" (n independent reimplementations) to "specification vs. engine correctness" (one shared, testable component). The compilation is deterministic and invertible; the evaluation rules are Backus's equations. The engine is the only component that must be correct.

6.2 Performance Characteristics

Forward chaining in AREST is incremental: when a command adds or modifies facts, only derivation rules whose antecedents reference affected predicates fire. The engine does not recompute the full population on each request.

The working set for a single command is bounded by the entity's participation in the schema. In the reference implementation, each entity instance occupies an independent storage unit. A command touches at most the target entity, its state machine, and entities reachable through the schema's roles. For schemas with k roles, constraint evaluation per command is $O(k)$, independent of total population size.

Aggregation rules (Insert/fold over a subset of the population) are an exception. Computing a derived sum over related entities requires traversing the relevant subset, which is proportional to the number of related entities, not the total population.

Facts are indexed by predicate and role player; lookup is constant-time per index. For FORML 2 derivation rules, which are stratifiable by construction (negation appears only in constraints, not in derivation rule heads), forward chaining to fixed point is polynomial in population size.

6.3 Debuggability

In a combinator-based system, traditional step-through debugging does not apply. AREST provides three diagnostic mechanisms:

1. **Constraint violations as readings.** When a constraint is violated, the engine returns the original FORML 2 text: "Each Customer has at most one Email." The specification is the error message.
2. **Derivation trace as facts.** Every derived fact is attributable to the derivation rule that produced it. The trace is queryable over the same population, using the same engine.
3. **Compiled model inspection.** The compiled AST is a data structure, not opaque bytecode. Developers inspect the combinator tree directly.

The trade-off is real: developers must learn to read combinator trees rather than stepping through imperative code. The benefit is that the artifact being inspected is the artifact being executed, not a separate debugging representation.

6.4 Induction and Ergonomics

Who writes the readings? The reference implementation provides an induction pipeline where a large language model translates natural language domain descriptions into structured claims (nouns, readings, constraints, transitions). The LLM's role is strictly translation; all reasoning is performed by the engine after ingestion. Malformed claims are rejected via CSDP validation with the violated reading as the error message, bounding the LLM's failure mode to rejected claims rather than silently incorrect programs.

For developers with ORM2 expertise, the LLM is unnecessary. Tools such as NORMA produce FORML 2 directly from graphical models. The induction pipeline is an accessibility layer, not an architectural dependency.

6.5 Expressiveness Boundary

FORML 2 can express entity and value types, n-ary fact types, subtypes, the full ORM2 constraint taxonomy, derivation rules with aggregation, and state machines. This covers the domain logic layer of most information systems.

Three categories of concern fall outside the model:

1. **Temporal logic.** Time-varying propositions beyond state machine sequencing (e.g., "X must occur within 30 days of Y") require temporal operators outside FOL.
2. **Side effects.** Payments, emails, and external API calls are not propositions and cannot be modeled as facts in the population.
3. **Partial failure.** Saga patterns, retry logic, and compensation are process concerns, not predicate logic.

These limitations are by design. AREST models what is true about the domain, not how to make it true operationally. Side effects are triggered at the transport boundary: a state machine transition fires within the population, and the transport layer dispatches the effect. If the effect fails, the transition does not commit. The boundary between declarative logic and imperative IO is explicit.

7 Related Work

Datalog [9] shares AREST's bottom-up, fixed-point evaluation. AREST differs in three respects: (1) FORML 2 provides a controlled natural language surface; (2) ORM2's constraint taxonomy is richer; (3) AREST compiles to combining forms and renders as hypermedia via REST. Unlike **Prolog**'s top-down evaluation with backtracking, AREST evaluates bottom-up, and ORM2 makes open-world vs. closed-world assumptions explicit per predicate.

Alloy [10] and **TLA+** [11] are specification languages with model checkers that verify properties of specifications but do not produce running systems. AREST specifications are running systems: the reading is simultaneously the specification and the executable. Alloy's relational logic is similar in expressiveness to ORM2's fact types, but Alloy models are artifacts separate from the implementations they describe.

GraphQL provides a query language and type system but does not close the specification–implementation gap: resolvers are hand-written functions that can diverge from the schema. **OData** standardizes REST query conventions but remains procedural. **RDF/Linked Data** represents knowledge as triples and supports inference via RDFS/OWL, but lacks the constraint taxonomy, state machines, and algebraic computation model that ORM2 and Backus's FP provide.

CQRS and Event Sourcing separate commands from queries and derive state by replaying an event stream. AREST shares this structure: commands transform the population (write side), representations project it (read side), and state machines are folds over event sequences (Section 3.2). The difference is that in AREST, events and derived state are both facts in the same population governed by the same constraints, and the fold is a compiled combinator rather than a hand-written projection function. Event sourcing leaves projection logic to the developer; AREST compiles it from the readings.

CRDT-based systems [13] model distributed state transitions algebraically and guarantee convergence without coordination. AREST and CRDTs both treat state algebraically, but AREST enforces constraints that may reject transitions, whereas CRDTs accept all operations and resolve conflicts structurally.

Proof-Carrying Code [12] verifies executable code against safety policies at the machine code level. AREST's Curry-Howard analogy operates at the domain level: proposing a fact type is analogous to proposing a theorem, and CSDP validation is the consistency checker.

AREST composes Backus's AST with Fielding's REST, two state transfer systems published 23 years apart in adjacent communities (programming languages and distributed systems) that operate at orthogonal layers of the same architecture.

8 Conclusion

AREST composes Applicative State Transition systems with Representational State Transfer. The readings are axioms. The engine applies functions to populations. The API renders the result as hypermedia. The client navigates by following links. Each link is a function application waiting to happen.

The theoretical foundations: first-order logic, Church's computation [2], Codd's relations [3], Backus's algebra [1], Halpin's modeling [5], Fielding's distribution [4]. AREST is their synthesis.

Acknowledgments

Claude (Anthropic) was used as a programming assistant during implementation of the reference system and as a drafting aid during preparation of this manuscript. All architectural decisions, theoretical contributions, and the composition of AST with REST were made by the human author.

References

[1] J. Backus, "Can Programming Be Liberated from the von Neumann Style? A Functional Style and Its Algebra of Programs," *Communications of the ACM*, vol. 21, no. 8, pp. 613–641, 1978.

[2] A. Church, "An Unsolvable Problem of Elementary Number Theory," *American Journal of Mathematics*, vol. 58, no. 2, pp. 345–363, 1936.

[3] E.F. Codd, "A Relational Model of Data for Large Shared Data Banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[4] R.T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Doctoral dissertation, University of California, Irvine, 2000.

[5] T. Halpin, *Information Modeling and Relational Databases*, 2nd ed. Morgan Kaufmann, 2008.

[6] T. Halpin, "Objectification and Atomicity," unpublished note, updated 2020.

[7] T. Halpin and M. Curland, "Automated Verbalization for ORM 2," in *OTM Workshops*, pp. 1181–1190, 2006.

[8] W.A. Howard, "The Formulae-as-Types Notion of Construction," in *To H.B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, Academic Press, pp. 479–491, 1980.

[9] S. Ceri, G. Gottlob, and L. Tanca, "What You Always Wanted to Know About Datalog (And Never Dared to Ask)," *IEEE Transactions on Knowledge and Data Engineering*, vol. 1, no. 1, pp. 146–166, 1989.

[10] D. Jackson, *Software Abstractions: Logic, Language, and Analysis*, revised ed. MIT Press, 2012.

[11] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[12] G.C. Necula, "Proof-Carrying Code," *Proc. 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pp. 106–119, 1997.

[13] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-Free Replicated Data Types," in *Stabilization, Safety, and Security of Distributed Systems (SSS)*, LNCS 6976, pp. 386–400, 2011.